



IMPLEMENT MISSING C++26 FEATURES IN CLANG

Author:
- Muhammad Bassiouni

Mentors:
- Vassil Vassilev
- Aaron Jomy



AGENDA

01 About Me

02 Papers in Motion

03 Brief About N3356

04 Brief About P1814R0

05 Current Status



- Bachelor of Artificial Intelligence (Class of 2025)
- Maintainer of LLVM C Library
- GSoC2026/LLVM Mentor (LLVM libc + compiler-rt)



Papers in Motion

C2y

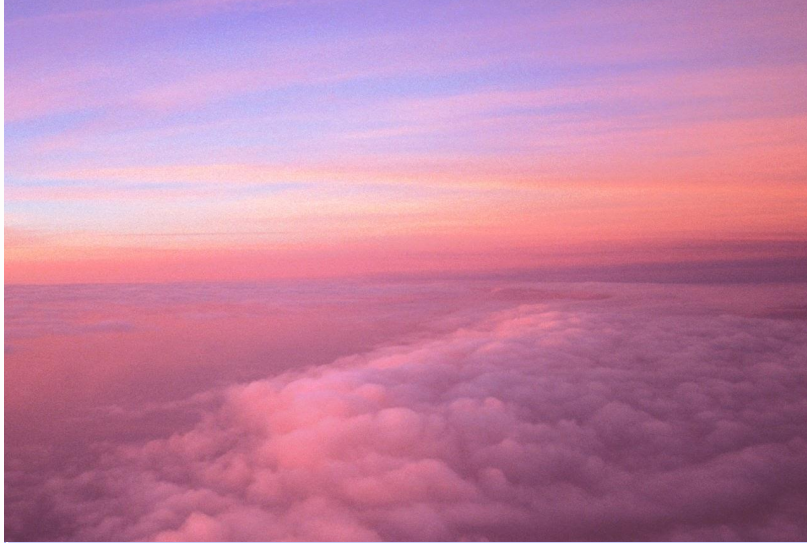
N3356

`if` declarations (if/switch).

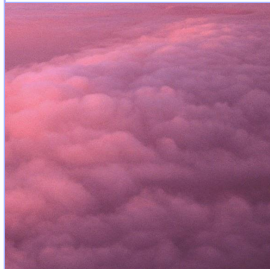
P1814R0

Class Template Argument Deduction
for Alias Templates.

C++20



Brief About Each Paper



Brief About N3356 (if/switch declaration)

01

Adds a single declaration

Adds a single declaration to the grammar of if/switch condition.

02

Similar to C++, but not the same

First clause of condition is a declaration.

Second clause of condition is only an expression, not a simple declaration.

03

Adds more declaration specifiers

Because of the nature of the first clause being a declaration.

```
1  if (bool x = true; x) {}
2  if (bool x = false) return x;
3  if ([[maybe_unused]] bool x = true) {}
4  if (bool x [[maybe_unused]] = true) {}
5  if ([[maybe_unused]] int x = 3; x > 0) {}
6  if (struct A { int x; } a = { .x = 1; }; a.x) {}
7  if (int arr[] = {1,2,3}; arr[1]) {}
8  if (auto x = 1; x) {}
9  if (static_assert(true); true) {}
10 if ([clang::assume(1 > 0)]; true) {}
11 if ([[]]; true) {}
12 if (__attribute__((assume(1 > 0))); true) {}
13 if (__attribute__(()); true) {}
14 if (__attribute__((deprecated)) auto x = 3) {}
15 if (auto x __attribute__((deprecated)) = 3) {}
16 if (__extension__ [[maybe_unused]] int x = 3; x > 0) {}

1  switch (int x [[maybe_unused]] = 1) {}
2  switch ([[maybe_unused]] int x = 1) {}
3  switch (__attribute__((assume(1 > 0))); 1) {default:}
4  switch (__attribute__(()); 1) {default:}
5  switch (__attribute__((deprecated)) auto x = 3) {default:}
6  switch (auto x __attribute__((deprecated)) = 3) {default:}
7  switch (__attribute__((deprecated)) auto x = 3) {default: x += 1;}
```

Brief About P1814R0 (CTAD for Alias Templates)

01

Adds template deduction for aliases

Extends CTAD support for aliases in C++20.

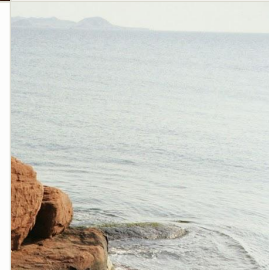
```
1 // before
2
3 template<typename T> struct A { A(T) {} };
4
5 template<typename T> using Proxy = T;
6 template<typename T> using C = Proxy<A<T>>;
7
8 C<int> test {42};
```

```
1 // after
2
3 template<typename T> struct A { A(T) {} };
4
5 template<typename T> using Proxy = T;
6 template<typename T> using C = Proxy<A<T>>;
7
8 C test {42};
```

Current Status

N3356

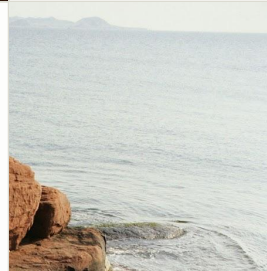
- Mostly done (see [PR#198244](#))
- Enabled in C89 (compatible extension)
- Declaration directives are handled properly (such as `__extension__`, attribute sequence declaration and `static_assert-decl`)
- Bug free, feature-complete implementation (ahead of GCC).



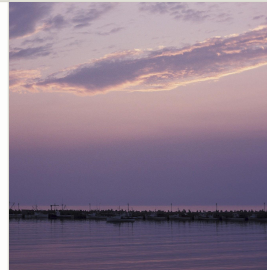
Current Status

P1814RO

- Basic implementation exist, but not stable
- Clang rejects valid code or crash in some cases
- Not enabled by default in the compiler
- Many reported issues which needs triage



THANK YOU



Feel free to ask questions or give feedback!

FIND ME AT

muhammad.m.bassiouni@gmail.com

github.com/bassiounix

linkedin.com/in/bassiounix